MiniShop - site e-commerce + back-office (SAE)

[![CI - GitHub (dev)](https://github.com/jojo8356/SAE-302/actions/workflows/ci.yml/badge.svg)](https://github.com/jojo8356/SAE-302/actions/workflows/ci.yml) [![CI - GitLab (prod)](https://gitlab.example.com/minishop/badges/main/pipeline.svg)](https://gitlab.example.com/minishop/-/pipelines)

Projet pédagogique : catalogue, panier, commandes, espace d'administration - **PHP 8 / MySQL (PDO) / procédures stockées / déclencheurs / architecture MVC** - avec sa base de données conçue, normalisée, testée, et son cahier des charges complet.

> **CI/CD dev vs prod** : le dev avant la prod se fait sur **GitHub** (`.github/workflows/ci.yml` - adaptation de `.gitlab-ci.yml`), la prod/notée reste sur **GitLab** (`.gitlab-ci.yml`). Les deux pipelines exécutent les mêmes 6 jobs (php-lint, style PSR-12, security-scan, base-tests mysql:8.0, unit phpunit, docs PlantUML) avec les mêmes images (`php:8.2`, `mysql:8.0`, `openjdk:17`). `concurrency/cancel-in-progress` reproduit `interruptible: true`.

> **Règle « pas de re-run »** : si la CI/CD est **verte avant le merge** (PR/MR au statut [x]), elle n'a **pas besoin d'être relancée après le merge**. Le run déclenché sur `main` par un merge est un **relais de traçabilité**, pas une vérification : mêmes 6 jobs, mêmes images, **même arbre de fichiers** (GitHub Actions exécute déjà le workflow `pull_request` sur le commit de fusion `refs/pull/<n>/merge`, GitLab CI sur le merge result). Un re-run manuel - `gh run rerun <id>`, bouton « Re-run jobs », « Retry » GitLab - ne peut donc rien révéler de neuf et coûte 5 à 8 min de machine (service `mysql:8.0` compris) par MR. On relance **uniquement** si `main` a avancé depuis le run de la PR, si le run a été **annulé** (`cancel-in-progress`), si le code a été corrigé puis poussé (c'est alors le `push` qui relance), ou en cas de **panne d'infrastructure** (miroir PlantUML, `apt`, registre Docker) - règle complète et tableau des 6 exceptions : **CDC §14.2**.

Cadre	SAE - BUT Informatique / Licence 3, Université Côte d'Azur
Groupe	3 étudiants (prénoms/noms/numéros dans §Membres ci-dessous)
Référente	Thanh-Phuong Nguyen - `thanh-phuong.nguyen@univ-cotedazur.fr`
Documents	[`docs/01-cahier-des-charges-MiniShop.md`](docs/01-cahier-des-charges-MiniShop.md) (spécification **et** conception de base de données) - [`docs/02-document-tests-validation.md`](docs/02-document-tests-validation.md) - [`docs/03-support-soutenance.md`](docs/03-support-soutenance.md) - [`docs/04-conception-bd-et-sql.md`](docs/04-conception-bd-et-sql.md) (livrable n°3 autonome : MCD, MLD, normalisation, analyse + **liens vers les scripts SQL livrés en fichiers**) - [`docs/05-wbs-projet.md`](docs/05-wbs-projet.md) (WBS : 6 divisions, 16 lots, 86 tâches individuelles, charges bottom-up, plan de charge, Gantt, chemin critique) - [`docs/08-fiche-suivi-commits.md`](docs/08-fiche-suivi-commits.md) (fiche de suivi des 32 commits + script de rejeu vers un nouveau GitLab)

Scripts SQL	[`sql/01_minishop_schema.sql`](sql/01_minishop_schema.sql) (DDL + données) - [`sql/02_minishop_procedures.sql`](sql/02_minishop_procedures.sql) (1 fonction + 17 procédures) - [`sql/03_minishop_triggers.sql`](sql/03_minishop_triggers.sql) (16 déclencheurs) - [`sql/04_minishop_demo.sql`](sql/04_minishop_demo.sql) (démo via `CALL`) - chaîne validée de zéro sur MariaDB 11.8.6
État	base de données + 29 tests SQL **exécutés avec succès** (règles RB-01...RB-20, vues, frais de port) ; application PHP **livrée** (MVC, design system, suite PHPUnit, E2E HTTP 180 vérifications) ; écarts restants suivis dans [`docs/10-todo-corrections.md`](docs/10-todo-corrections.md)

Prérequis

- PHP **8.2+** avec les extensions `pdo_mysql`, `mbstring`, `json`, `intl`
- MySQL **8.0.16+** (les contraintes `CHECK` sont appliquées depuis cette version) **ou MariaDB 10.6+**
- Apache 2.4 avec `mod_rewrite` (ou le serveur embarqué PHP pour un essai rapide)
- client `mysql` / `mariadb` en ligne de commande, `bash`
- facultatif : `pandoc` + `plantuml` (rendu des documents et des diagrammes), `phpunit`, `phpcs`

Installation (5 commandes)

```
# 1. cloner et placer le projet
git clone <URL_DU_DEPOT> minishop && cd minishop

# 2. configuration (jamais commitée)
cp app/Config/env.example.php app/Config/env.php      # puis éditer db_user / db_pass

# 3. créer la base, les tables, les 16 triggers, la fonction + 17 procédures et le jeu de
démonstration
DB_USER=root DB_PASS='...' ./scripts/load_db.sh

# 4. vérifier que la base est saine (29 tests de règles de gestion, vues et frais de port)
DB_USER=root DB_PASS='...' ./scripts/run_sql_tests.sh   # 29 tests, attendu : « Tests conformes :
29 / 29 »

# 5. lancer (développement) - en production : DocumentRoot = public/
php -S localhost:8000 -t public
#   http://localhost:8000/            (front-office)
#   http://localhost:8000/admin        (back-office, compte admin requis)
```

Le script `load_db.sh` encapsule les fichiers contenant des procédures/triggers avec un `DELIMITER $$` (le client `mysql` n'accepte pas `DELIMITER` en mode batch). Sous MySQL Workbench ou phpMyAdmin, ouvrir et exécuter `sql/01 -> 02 -> 03 -> 04` dans cet ordre fonctionne aussi.

Comptes de démonstration

Rôle	Email	Mot de passe (démo uniquement)
Client	`alice@example.com`	`Demo2026!`

Client	`bruno@example.com`	`Demo2026!`
Client	`carla@example.com`	`Demo2026!`
Administrateur (rôle `SUPER`)	`admin@minishop.fr`	`Admin2026!`

Les mots de passe sont stockés **hachés** (`password_hash()` bcrypt cost 12) dans `sql/01_minishop_schema.sql`. Pour en régénérer : `php -r 'echo password_hash("MonMotDePasse", PASSWORD_BCRYPT, ["cost"=>12]), "\n";'`. Ne rejouez **pas** `sql/04_minishop_demo.sql` sur une installation exposée (commandes de démonstration) ; `php scripts/deploy.php --write-sql --db` régénère des **hashes bcrypt aléatoires** et des **emails de démo uniques** (domaine `.invalid`, suffixe aléatoire) pour la démo publique (le script rend la version `.sh` historique obsolète, celle-ci a été supprimée).

Contenu de la base (vérifiable en 1 requête)

```
SELECT 'tables' objet, COUNT(*) n FROM information_schema.tables    WHERE table_schema =
DATABASE()
UNION ALL SELECT 'procedures', COUNT(*) FROM information_schema.routines WHERE routine_schema =
DATABASE() AND routine_type = 'PROCEDURE'
UNION ALL SELECT 'triggers',   COUNT(*) FROM (SELECT DISTINCT TRIGGER_NAME FROM
information_schema.TRIGGERS WHERE TRIGGER_SCHEMA = DATABASE()) t;
```

8 tables - 3 vues - 1 fonction + 17 procédures stockées - 16 déclencheurs - 12 produits - 4 catégories - 3 clients - 1 admin - 4 commandes de démonstration (dont une qui paie 4,90 EUR de frais de port).

État réel du dépôt (à lire avant de chercher un fichier)

Zone	Statut	Comment le vérifier
`app/**`, `public/**`, `data/`, `scripts/seed.php`	**application livrée et testée** (PHP pur, moteur JSON, **design system CSS/JS + SVG**)	`npm run test:all` -> 7 suites vertes (108 + 28 + 48 + 34 + 180 vérifications)
`app/Model/Data/`	moteur « JSON-comme-SGBD » : schém contraintes (NOT NULL/ENUM/CHECK/UNIQUE/FK), trigger transactions, vues, journal JSONL - piloté par l'énumération `StorageDriver` (JSON actif ; `SqlStore` = pilote PDO réel, procédures à brancher)	`node scripts/wasm_run.mjs tests/php/engine_smoke.php`
`tests/php/**`, `scripts/wasm-e2e.mjs`, `scripts/run-all-tests.mjs`	**livrés et exécutés** (voir « Tests livrés » ci-dessus)	`npm run test:all`
`sql/`, `tests/sql/`, `docs/` (CDC, tests, soutenance, annexes, diagrammes)	**livrés** ; `sql/` = référence comportementale du moteur JSON	`./scripts/run_sql_tests.sh` -> 29/29
`tests/security/controles.sh`, `tests/charge/reserver.sh`, `tests/perf/mesurer.sh`	**scripts livrés** ; les deux derniers supposent l'application en service	`bash -n` passe

`tests/sql/out/*.log`	non versionnés (régénérables) - seul `tests/sql/rapport_tests_sql.md` est versionné, preuve de recette à l'appui	`.gitignore`, §14.1 règle 7

Tests livrés

L'application est livrée avec **sept suites, toutes vertes**, exécutables d'un bloc :

```
npm run test:all        # ou : node scripts/run-all-tests.mjs
```

> **Sécurité** : la note complète « failles testées -> corrections -> code » (S-01...S-10, SEC-01...SEC-13, limites assumées) est dans **`docs/07-note-cybersecurite.md`** ; les attaques correspondantes sont rejouées en continu par les batteries T6/T12/T13 de la suite E2E.

Suite	Fichier	Couverture	Verdict
Linter strict types	`scripts/strict_types_lint.php`	règle `declare_strict_types` de php-cs-fixer + types natifs « PHPStan level 9 » (params, retours, propriétés) + **règle maison : opérateur ternaire interdit** (`? :` et `?:`, remplacés par `if/else` ; `??` et types nullables `?int` autorisés) sur **tout le livrable PHP**	[x] 0 violation / 86 fichiers
Moteur JSON	`tests/php/engine_smoke.php`	contraintes, triggers, transactions, journal du moteur	[x] vert
Règles de gestion	`tests/php/run_tests.php`	T-01...T-29 du document de tests (RB-01...RB-20, vues, frais de port)	[x] 29/29
Unitaires	`tests/php/unit_tests.php`	`Text` (fold/slug/collation), échappement, CSRF, **matrice RB-11 complète** (36 couples), `Filter`, `PanierSession`, prix TTC, journal	[x] 98/98
Fonctionnalités	`tests/php/features_test.php`	**UC-01...UC-14** via les repositories (catalogue, compte, panier, commande, annulation, back-office, indicateurs)	[x] 108/108
Contrats de design	`tests/php/css_contract.php`	tokens de `docs/DA.md`, contrastes WCA calculés, classes des vues réellement définies, `prefers-reduced-motion`, feuille d'impression	[x] 28/28
Contrat responsive	`tests/php/responsive_contract.php`	320 px -> 1536 px : paliers en `em`, menu repliable, tableaux accessibles, panier en cartes, `dvh`, encoches, `pointer: coarse`, container queries	[x] 48/48
Unitaires PHPUnit	`tests/phpunit/`	`Text`, `Filter`, moteur JSON (contraintes, transactions), matrice RB-11, prix TTC, contrats de sécurité - exécutés hors ligne par `run-local.php`	[x] 34/34

E2E HTTP	`scripts/wasm-e2e.mjs`	T1...T14 : parcours réels (cookies + CSRF) - pages publiques, auth, commande 488,40 EUR, annulation, back-office, cloisonnement, catalogue avancé, compte, panier (plafond/re-tri/prix falsifié), CRUD admin, filtres + audit + journal, **sécurité offensive** (injection, XSS stocké, IDOR, fixation de session, traversées, escalation), rôles RB-13, contact/API/paramètres	[x] 180/180

Commandes individuelles : `npm test` (règles), `npm run test:units`, `npm run test:features`, `npm run test:design`, `npm run test:phpunit`, `npm run test:e2e` (re-seed puis parcours HTTP ; l'E2E **po llue** `data/minishop` par construction - re-seeder ensuite : `node scripts/wasm_run.mjs scripts/seed.php`).

Les suites PHP sont **autonomes** (aucun SQL, magasins JSON temporaires par section) : elles s'exécutent telles quelles avec `php tests/php/<suite>.php` sur une machine normale, ou via `node scripts/wasm_run.mjs` en sandbox.

Qualité de code type « Packagist » : sur machine normale (avec Composer), `composer cs:check` (php-cs-fixer, config `.php-cs-fixer.php`) et `composer lint:types` (PHPStan level 9 + strict-rules, config `phpstan.neon`) - tous deux en `require-dev`, **aucune dépendance runtime**. Dans la sandbox (Packagist injoignable), `npm run lint:types` rejoue les mêmes vérifications via le tokenizer PHP. Passages du 28/09 : 4 fermetures non typées corrigées, puis **éradication des 156 opérateurs ternaires du livrable** (vues, contrôleurs, dépôts, moteur, tests) remplacés par des `if/else` - règle maison ajoutée au linter.

Compléments livrés par ailleurs : `bash tests/security/controles.sh` (contrôles statiques), `bash tests/charge/reserver.sh` (survente), `bash tests/perf/mesurer.sh` (p95, ENF-01), `docs/diagrams/render.sh` (14 diagrammes), `./scripts/build-docs.sh` (assemblage documentaire complet).

Historique : la campagne SQL d'origine (`./scripts/run_sql_tests.sh`, 29/29, rapport `tests/sql/rapport_tests_sql.md`) reste jouable - `sql/` sert de référence comportementale au moteur JSON.

Arborescence

```
app/Config, app/Controller, app/Model, app/Repository, app/Security, app/View
docs/{01-cahier-des-charges, 02-document-tests-validation, 03-support-soutenance, 04-conception-
bd-et-sql, 05-wbs-projet, 06-todolist}, docs/annexes, docs/diagrams/{src/*.puml, *.png}
public/{index.php, .htaccess, css, js, img, gen_volumes.php, mesurer.php (dev-only, navigateur)}
scripts/{load_db.sh, run_sql_tests.sh, deploy.php, build-docs.sh, gen_volumes.php (wrapper dev)}
sql/{01_minishop_schema.sql, 02_minishop_procedures.sql, 03_minishop_triggers.sql,
04_minishop_demo.sql}
tests/{sql/** (manifest, fixture, tNN_*.sql, rapport), security/controles.sh, charge/reserver.sh,
perf/mesurer_sql.sh, perf/mesurer.php->public/mesurer.php, php/**}
var/log  -  .gitignore  -  .gitlab-ci.yml (6 jobs, prod)  -  .github/workflows/ci.yml (6 jobs,
dev/GitHub)
```

Documents et rendus (les diagrammes ne s'affichent PAS dans un aperçu sans réseau)

> **Si tu ne vois pas `dist/`** : ce dossier est listé dans `.gitignore` (artefact régénérable, §14.1) et donc masqué par certains explorateurs/aperçus. Une **copie visible** est maintenue dans **`livrables/`** à la racine (`livrables/*.docx`, `livrables/html/*.html`), régénérée à chaque `bash scripts/build-docs.sh`.

`bash scripts/build-docs.sh` produit, dans `dist/` (répertoire généré, hors Git) :

Rendu	Chemin	Image des diagrammes
Markdown (source de vérité, éditable)	`docs/01-...-MiniShop.md`	**chemins relatifs** -> non affichés dans un visualiseur sandboxé sans accès réseau ; ouvrir le `.docx` ou le `.html` pour les voir
Word	`dist/01-cahier-des-charges-MiniShop.docx`	images **intégrées** (14 figures)
HTML autonome	`dist/html/01-cahier-des-charges-MiniShop.html`	images **intégrées en base64** - lisible hors ligne, projetable, aucune dépendance
PNG / SVG	`docs/diagrams/*.{png,svg}`	sources rendues par `docs/diagrams/render.sh` (PlantUML)

Si un aperçu affiche une image cassée là où il y a un `![...](docs/diagrams/...png)`, le fichier existe : vérifier avec `ls docs/diagrams/*.png \| wc -l` (attendu **14**) et ouvrir `dist/html/01-cahier-des-charges-MiniShop.html`.

Membres et rôles (traçabilité GitLab)

Personne	Rôle (RACI du CDC §3.3)	Zones principales	Commi
Étudiant-e A	chef de projet - back-office	`app/Controller/`, `README.md`, `.gitlab-ci.yml`	>= 20
Étudiant-e B	modèle et base de données	`sql/`, `app/Repository/`, `tests/sql/`	>= 20
Étudiant-e C	présentation, sécurité, tests	`app/View/`, `public/js/`, `docs/02-*`	>= 20

Chaque personne committe aussi dans les deux autres zones (au moins 5 commits chacun) : `git shortlog -sne main` doit montrer une participation équilibrée, c'est un critère de notation individuel.

Sauvegarde / restauration (à faire avant une démo)

```
mysqldump --single-transaction --routines --triggers -u root -p minishop > backup.sql   #
--routines --triggers : OBLIGATOIRE ici
mysql -u root -p minishop < backup.sql
```

Sans `--routines --triggers`, une restauration « normale » perd la fonction, les 17 procédures et les 16 déclencheurs : les règles `RB-03`, `RB-06`, `RB-11`, `RB-18` cesseraient silencieusement d'être

appliquées.

Licence

Projet pédagogique - code et documents fournis tels quels, usage pédagogique uniquement.